

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks 50 Duration :60 Minutes Annexure A
Constraint-Based Problem Solving

Code of Conduct:

I __Harshavardhan .RV (192472163)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: HARSHAVARDHAN. RV

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	

Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a face recognition system on low-power edge devices (e.g., Raspberry Pi). The system must:

- A. Process live video streams in real time (<30 FPS latency)**
- B. Work under varying lighting conditions**
- C. Use limited memory and CPU resources**
- D. Detect and recognize faces from a predefined dataset**

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

ANS: Proposed System

A real-time face recognition system can be designed on a Raspberry Pi using **OpenCV** with a lightweight face detector and a compact recognition model.



2. Image Acquisition

- Use a USB camera or Raspberry Pi Camera.
- Capture continuous video frames.
- Process frames at approximately 640×480 resolution.
- To reduce CPU usage, process every 2nd frame if necessary while displaying the latest result.

Code:

```
cap = cv2.VideoCapture(0)
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

This provides a continuous live video stream.

3. Preprocessing

Each captured frame is preprocessed before detection.

Steps:

1. Resize the frame to a smaller resolution.
2. Convert RGB/BGR image to grayscale.
3. Apply histogram equalization or **CLAHE**.
4. Apply light Gaussian filtering to reduce noise.

Original Frame



Resize



Grayscale



CLAHE



Enhanced Frame

Why?

Grayscale reduces computation, while CLAHE improves visibility when the environment has shadows, low light, or uneven illumination.

4. Face Detection

For Raspberry Pi, a lightweight detector should be selected instead of a large deep-learning model.

Possible OpenCV choices:

- Haar Cascade — very lightweight and easy to deploy.
- LBP Cascade — faster than many traditional detectors.
- OpenCV DNN face detector — better accuracy but requires more computational resources.

For a low-power Raspberry Pi, **Haar/LBP Cascade** is a practical choice when the environment is controlled.

```
face_cascade = cv2.CascadeClassifier(
```

```
"haarcascade_frontalface_default.xml"  
)
```

```
faces = face_cascade.detectMultiScale(  
    gray,  
    scaleFactor=1.1,  
    minNeighbors=5,  
    minSize=(60, 60)  
)
```

5. Face Alignment and Cropping

After detecting a face:

Detected Face



Crop Face Region



Resize to Fixed Size



Normalize

For example, every face can be resized to **100 × 100 pixels**.

Alignment can also be performed using the eyes as reference points. This makes recognition more reliable when the person is slightly rotated.

6. Feature Detection / Extraction

The system needs to convert the detected face into numerical features.

For a very lightweight OpenCV implementation, **LBPH (Local Binary Patterns Histograms)** is suitable.

LBPH captures local texture patterns of the face and creates a histogram representation.

```
recognizer = cv2.face.LBPHFaceRecognizer_create()
```

```
recognizer.train(face_images, labels)
```

An alternative for better recognition accuracy is a **small pre-trained face embedding model**, but it requires more memory and CPU than LBPH.

7. Face Recognition

The extracted face is compared against the predefined training dataset.

```
label, confidence = recognizer.predict(face)
```

The system then applies a confidence threshold:

Recognized Face



Compare with Dataset



Confidence acceptable?



YES



NO



Person Name



UNKNOWN

For example:

```
if confidence < threshold:
```

```
    name = names[label]
```

```
else:
```

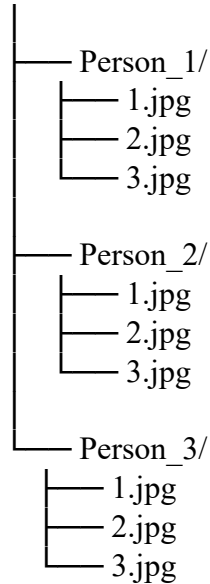
```
    name = "Unknown"
```

The threshold must be experimentally selected using the actual dataset and lighting conditions.

8. Predefined Dataset

The dataset can contain several images for each registered person:

dataset/



Using multiple images under different expressions and lighting conditions improves recognition.

9. How the Design Satisfies the Constraints

Constraint	Design Solution
Real-time processing	Resize frames, lightweight OpenCV detector, process selected frames
<30 FPS latency requirement	Low-resolution processing and lightweight LBPH/Haar pipeline reduce computation
Varying lighting	Grayscale + CLAHE/histogram normalization
Limited memory	Small images, compact dataset and lightweight models
Limited CPU	Avoid large neural networks; use OpenCV's optimized operations
Predefined faces	Train recognizer using registered people's images
Unknown persons	Confidence threshold identifies unregistered faces
Live video	OpenCV VideoCapture() continuously reads camera frames

Conclusion

The proposed **OpenCV-based edge face recognition system** is suitable for a Raspberry Pi because it uses a lightweight processing pipeline. **Frame resizing and selective frame processing** reduce CPU consumption, while **grayscale conversion and CLAHE** improve performance under different lighting conditions. **Haar/LBP face detection and LBPH**

recognition require considerably fewer resources than large deep-learning models. A predefined dataset allows the system to recognize registered people and classify other faces as **Unknown**. Thus, the design provides a practical balance between **real-time performance, recognition accuracy, memory usage, and low-power edge deployment**.

2. Automated Video Surveillance with Alert System

A security firm needs a video surveillance system that:

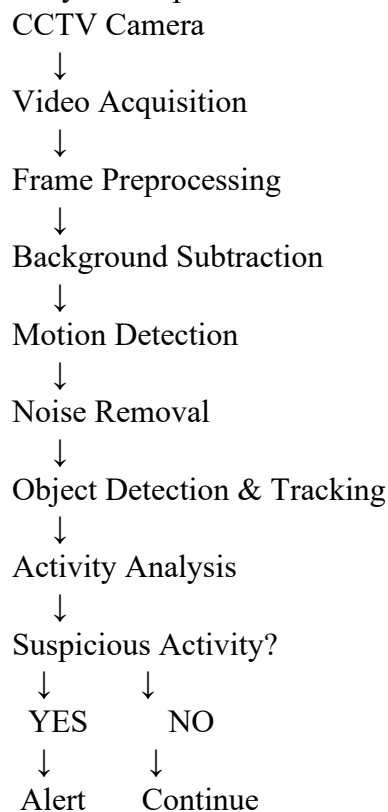
- A. Detects motion and unusual activities
- B. Generates alerts in real time
- C. Works with standard CCTV video feeds
- D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

Ans: Automated Video Surveillance with Alert System

An OpenCV-based surveillance system can monitor CCTV video continuously, detect moving objects, analyze their movement, and generate alerts when suspicious activity is detected.

1. System Pipeline



2. Video Acquisition

OpenCV's VideoCapture() can receive video from standard CCTV cameras, IP cameras, RTSP streams, or recorded videos.

```
cap = cv2.VideoCapture("cctv_video.mp4")
```

while True:

```
ret, frame = cap.read()
if not ret:
    break
```

3. Preprocessing

The captured frame is resized and converted to grayscale. Gaussian blur is applied to reduce noise.

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
gray = cv2.GaussianBlur(gray, (5,5), 0)
```

Benefits:

- Reduces camera noise.
- Reduces processing time.
- Improves motion detection.

4. Motion Detection

OpenCV's MOG2 background subtraction can separate moving objects from the background.

```
bg = cv2.createBackgroundSubtractorMOG2(
    history=500,
    varThreshold=40,
    detectShadows=True
)
```

```
mask = bg.apply(gray)
```

The system compares the current frame with the background and produces a motion mask.

5. Noise Removal

Small unwanted regions are removed using morphological operations.

```
kernel = cv2.getStructuringElement(
    cv2.MORPH_ELLIPSE, (5,5)
)
```

```
mask = cv2.morphologyEx(
    mask, cv2.MORPH_OPEN, kernel
)
```

```
mask = cv2.morphologyEx(
    mask, cv2.MORPH_CLOSE, kernel
)
```

- Opening removes small noise.
- Closing fills small gaps in objects.

6. Object Detection and Tracking

Contours are extracted from the motion mask.

```
contours, _ = cv2.findContours(
    mask,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)
```

```

for c in contours:
    if cv2.contourArea(c) > 1000:
        x, y, w, h = cv2.boundingRect(c)
        cv2.rectangle(
            frame, (x,y), (x+w,y+h), (0,255,0), 2
        )

```

The detected object's position, size, direction, and movement duration can be tracked.

7. Unusual Activity Detection

The system should not generate an alert for every movement. It can identify events such as:

Person enters restricted area



Remains for several seconds



Suspicious Activity



Generate Alert

Other examples include sudden movement, unusual direction, or excessive crowding.

8. Alert Generation

When suspicious activity is confirmed, the system can save the frame and generate an alert.

```

if suspicious_activity:

```

```

    cv2.imwrite("alert.jpg", frame)
    print("ALERT: Suspicious Activity!")

```

The system can also trigger a buzzer or send an alert to a monitoring application.

9. Handling Dynamic Backgrounds

Dynamic backgrounds such as moving trees, rain, shadows, and changing lighting can cause false detections.

This is handled using:

- Adaptive MOG2 background modeling.
- Gaussian filtering.
- Shadow detection.
- Minimum object-area threshold.
- Multi-frame confirmation.

Dynamic Background



Adaptive Background Model



Noise Filtering



Multi-frame Verification



Fewer False Positives

10. Real-Time Optimization

To maintain real-time performance:

- Use 640×480 resolution.

- Process every second frame if required.
- Analyze only important Regions of Interest (ROI).
- Use lightweight OpenCV operations.
- Save video only when suspicious activity occurs.

11. Constraint Justification

Requirement	Solution
Motion detection	MOG2 + contour detection
Unusual activity	Movement, duration and restricted-zone analysis
Real-time alerts	Immediate event-based alert
CCTV compatibility	OpenCV VideoCapture()
Noise handling	Gaussian blur + morphology
Dynamic backgrounds	Adaptive MOG2
Fewer false positives	Area threshold + multi-frame confirmation
Real-time processing	Lower resolution + ROI + frame skipping

Conclusion

The proposed OpenCV surveillance system uses video acquisition, preprocessing, adaptive background subtraction, motion detection, object tracking, and activity analysis. Noise is reduced using Gaussian filtering and morphological operations, while MOG2 handles dynamic backgrounds. Multi-frame verification and area thresholds reduce false positives. Frame resizing, ROI processing, and lightweight OpenCV operations make the system suitable for real-time CCTV surveillance and alert generation.

3. OCR System for Low-Quality Documents

A digitization company needs an OCR system that:

A. Extracts text from low-resolution, noisy scanned images

B. Handles varying fonts and illumination

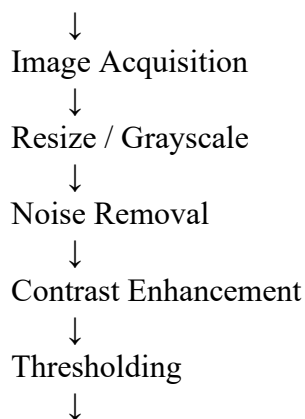
C. Works efficiently for batch processing

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

Ans: An OpenCV-based OCR system can improve poor scanned documents before sending them to an OCR engine such as Tesseract. The main goal is to remove noise, improve text visibility, separate text from the background, and process documents efficiently in batches.

1. Overall Pipeline

Scanned Document



Morphological Processing



Text Segmentation



Deskew / Alignment



OCR Engine



Recognized Text

2. Image Acquisition

The scanned document is loaded using OpenCV.

```
import cv2
```

```
img = cv2.imread("document.jpg")
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Grayscale reduces the image to one channel and decreases processing requirements.

3. Image Enhancement

Low-resolution documents may contain blurred or unclear characters. The image can be resized before OCR.

```
gray = cv2.resize(  
    gray, None,  
    fx=2, fy=2,  
    interpolation=cv2.INTER_CUBIC  
)
```

Benefit: Enlarging characters makes small text easier for the OCR engine to recognize.

4. Noise Removal

Scanned documents may contain dots, scratches, and compression noise.

```
blur = cv2.GaussianBlur(gray, (5,5), 0)
```

For salt-and-pepper noise, median filtering can also be used:

```
clean = cv2.medianBlur(gray, 3)
```

Benefit: Removes unwanted noise while preserving important character boundaries.

5. Illumination and Contrast Enhancement

Uneven lighting can make some parts of a document darker than others.

CLAHE can improve local contrast:

```
clahe = cv2.createCLAHE(  
    clipLimit=2.0,  
    tileGridSize=(8,8)  
)
```

```
enhanced = clahe.apply(clean)
```

Benefit: Makes faint characters more visible even when illumination is uneven.

6. Thresholding / Segmentation

The enhanced image is converted into a binary image where text and background are separated.

```
binary = cv2.adaptiveThreshold(  
    enhanced,
```

```

255,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY,
31,
15
)

```

Adaptive thresholding is useful for documents with different brightness levels.

Gray Image

↓

Adaptive Threshold

↓

Black Text + White Background

7. Morphological Processing

Small gaps in characters can be repaired using morphological operations.

```

kernel = cv2.getStructuringElement(
    cv2.MORPH_RECT, (2,2)
)

```

```

processed = cv2.morphologyEx(
    binary,
    cv2.MORPH_CLOSE,
    kernel
)

```

Benefit:

- Removes small gaps.
- Connects broken character regions.
- Improves character shape for OCR.

8. Text Segmentation

The document can be divided into meaningful regions such as:

Document

↓

Paragraphs

↓

Lines

↓

Words / Characters

Contours can help identify text regions:

```

contours, _ = cv2.findContours(
    processed,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

```

This allows the OCR engine to process relevant text regions instead of unnecessary background areas.

9. Deskewing

Scanned pages may be slightly rotated.

Tilted Document



Detect Text Orientation



Rotate Image



Straight Document

Deskewing is important because horizontally aligned text is easier for OCR engines to recognize.

10. OCR Recognition

After preprocessing, the cleaned image is passed to an OCR engine such as Tesseract.

```
import pytesseract
```

```
text = pytesseract.image_to_string(processed)
```

```
print(text)
```

The OpenCV pipeline improves the input before recognition.

11. Handling Different Fonts

Different fonts can have different character shapes. The system improves recognition by:

- Maintaining character boundaries through appropriate thresholding.
- Avoiding excessive blurring.
- Using adaptive thresholding.
- Providing a clean, aligned image to the OCR engine.

For documents containing many font styles, an OCR engine with suitable language/font models should be selected.

12. Batch Processing

For a large number of documents, each file can be processed automatically.

Documents Folder



Read Image 1



Preprocess



OCR



Save Text



Read Image 2



Preprocess



OCR



Save Text



...

This avoids manually processing every scanned document.

13. Constraint Justification

Requirement	OpenCV Solution	Benefit
Low-resolution images	Resize + interpolation	Improves small characters
Noise	Median/Gaussian filtering	Removes unwanted noise
Varying illumination	CLAHE	Improves local contrast
Text/background separation	Adaptive thresholding	Handles uneven brightness
Broken characters	Morphological closing	Connects character regions
Skewed documents	Deskewing	Aligns text correctly
Different fonts	Careful enhancement + OCR models	Preserves character structure
Batch processing	Automated image loop	Efficient for large datasets

Conclusion

The proposed OCR system uses OpenCV preprocessing, enhancement, thresholding, morphology, segmentation, and deskewing before sending the document to an OCR engine. Resizing improves low-resolution text, filtering removes noise, CLAHE handles uneven illumination, and adaptive thresholding separates text from the background. Morphological operations repair broken characters, while segmentation and deskewing improve text recognition. The complete pipeline can be automated for efficient batch processing of large numbers of low-quality scanned documents.

4. Facial Expression Recognition for Mobile Application

A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints: A. Limited processing power (mobile CPU)

B. Real-time performance

C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

Ans: 4. Facial Expression Recognition for Mobile Application

A lightweight OpenCV-based system can detect a face from the mobile camera and classify the expression as **Happy, Sad, or Neutral**. The system should use a small number of facial features and lightweight processing to maintain real-time performance.

1. Overall Pipeline

Mobile Camera



Frame Acquisition



Resize + Grayscale



Lighting Normalization



Face Detection



Face Alignment



Feature Extraction



Expression Classification



Happy / Sad / Neutral

2. Image Acquisition

The mobile camera continuously captures frames.

```
cap = cv2.VideoCapture(0)
```

```
ret, frame = cap.read()
```

For mobile devices, the frame can be resized to around **320 × 240** or **640 × 480** to reduce CPU usage.

3. Preprocessing

The captured frame is converted to grayscale and lightly filtered.

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
gray = cv2.GaussianBlur(gray, (3,3), 0)
```

Benefits:

- Grayscale reduces computation.
- Filtering reduces camera noise.
- Lower resolution improves processing speed.

4. Face Detection

OpenCV's **Haar Cascade** detector can be used because it is lightweight and suitable for mobile CPUs.

```
face_cascade = cv2.CascadeClassifier(  
    "haarcascade_frontalface_default.xml"  
)
```

```
faces = face_cascade.detectMultiScale(  
    gray,  
    scaleFactor=1.1,  
    minNeighbors=5,  
    minSize=(60,60)  
)
```

The detected face is cropped from the image.

Camera Frame



Face Detection



Face Region



Expression Analysis

5. Handling Lighting Changes

Lighting variations can affect facial features. **Histogram equalization or CLAHE** can improve the face image.

```
clahe = cv2.createCLAHE(  
    clipLimit=2.0,  
    tileGridSize=(8,8)  
)
```

```
face = clahe.apply(face)
```

This makes facial features more consistent under bright or dim lighting.

6. Handling Slight Head Movement

The detected face can be aligned using facial landmarks such as:

- Eyes
- Nose
- Mouth

The face is then resized to a fixed size.

Moving Face



Detect Face



Locate Eyes / Mouth



Align Face



Fixed-Size Face

This reduces the effect of small rotations and movements.

7. Feature Extraction

Instead of processing the complete image with a large model, lightweight features can be extracted.

Useful features include:

- Mouth width and height
- Eye openness
- Eyebrow position
- Mouth curvature
- Facial texture

For example:

Happy → Mouth corners raised / wider smile

Sad → Downward mouth shape / lowered features

Neutral → Relatively relaxed mouth and eyes

LBP (Local Binary Pattern) features can also be used because they are computationally inexpensive and capture facial texture.

8. Expression Classification

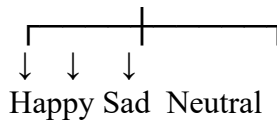
The extracted features are passed to a lightweight classifier.

Facial Features



Classifier





A small **SVM** or decision-tree classifier can be trained using facial-expression images.

9. Real-Time Processing

The system does not need to perform every operation on every camera frame.

Camera → 30 FPS

↓
Process selected frames

↓
Expression result

↓
Display result

For example, face detection can be performed periodically, while tracking can be used between detections.

This reduces CPU consumption while maintaining a smooth user experience.

10. Accuracy and Efficiency Balance

Requirement	Solution
Limited CPU	Grayscale + low-resolution frames
Real-time performance	Lightweight Haar detector and simple features
Lighting changes	CLAHE / histogram normalization
Head movement	Face alignment and tracking
Expression recognition	LBP + lightweight classifier
Memory limitation	Small model and fixed-size face images
Stable results	Analyze multiple consecutive frames

11. Reducing Incorrect Predictions

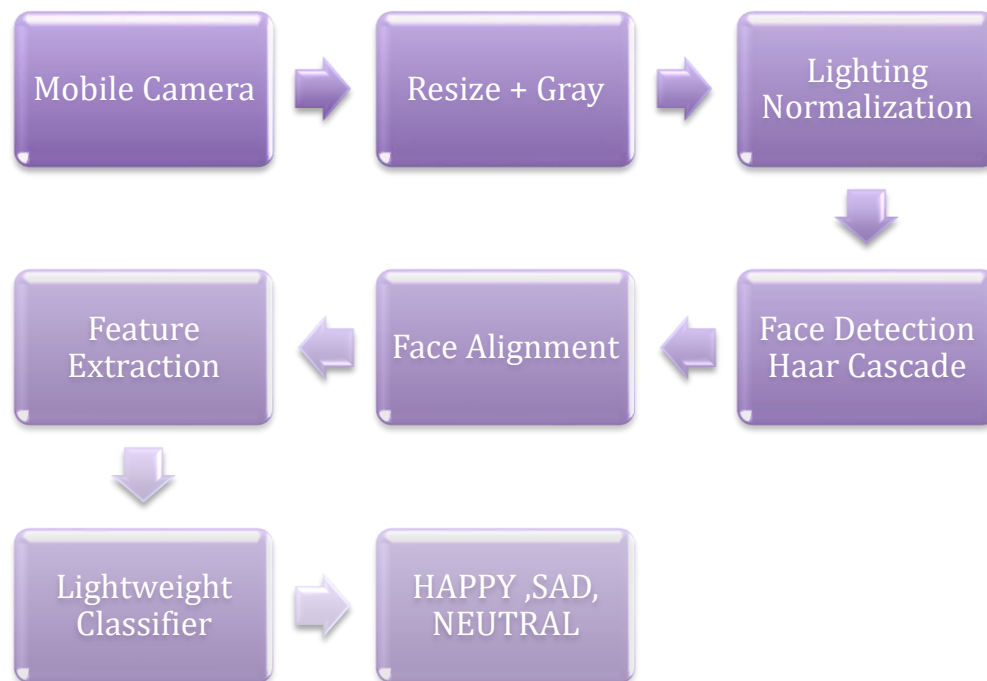
A single frame may produce an incorrect expression. Therefore, the system can use **temporal smoothing**.

Frame 1 → Happy
Frame 2 → Happy
Frame 3 → Neutral
Frame 4 → Happy
Frame 5 → Happy

↓
Final: Happy

The most frequently detected expression over several frames is selected.

12. Complete Architecture



Conclusion

The proposed system balances accuracy and efficiency by using OpenCV's lightweight face detection, grayscale preprocessing, CLAHE for lighting normalization, face alignment for small head movements, and simple facial features with a lightweight classifier. Processing at a reduced resolution and using frame selection or tracking lowers CPU usage. Finally, temporal smoothing prevents unstable predictions. Therefore, the system is suitable for real-time facial expression recognition on mobile devices with limited processing power.

5. Smart Traffic Monitoring System

A city authority needs a system to:

- A. Detect vehicles from live video streams
- B. Track movement across frames
- C. Count vehicles and display statistics
- D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

Ans: **Smart Traffic Monitoring System**

A smart traffic monitoring system can use **OpenCV** to detect vehicles from live CCTV video, track their movement, count vehicles, and display traffic statistics in real time.

1. Overall Pipeline

Live CCTV Video



Frame Acquisition



```

Preprocessing
↓
Vehicle Detection
↓
Vehicle Tracking
↓
Counting Line
↓
Vehicle Count
↓
Statistics + Visualization

```

2. Video Handling

OpenCV's VideoCapture() reads the live CCTV stream.

```

import cv2
cap = cv2.VideoCapture("traffic.mp4")
while True:
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imshow("Traffic", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()

```

The same approach can be used with an IP/RTSP camera.

3. Preprocessing

To improve speed, each frame can be resized and converted to grayscale.

```

frame = cv2.resize(frame, (640, 480))
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

```

Gaussian blur can reduce small noise:

```

gray = cv2.GaussianBlur(gray, (5,5), 0)

```

4. Vehicle Detection

For a lightweight OpenCV solution, **background subtraction** can identify moving vehicles.

```

bg = cv2.createBackgroundSubtractorMOG2(
    history=500,
    varThreshold=40
)
mask = bg.apply(frame)

```

Contours are then extracted from the foreground mask.

```

contours, _ = cv2.findContours(
    mask,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

```

Very small contours are ignored to reduce false detections.

5. Vehicle Tracking

Each detected vehicle is assigned an ID and its position is tracked across frames.

Frame 1 → Vehicle ID 1 → (100,200)

Frame 2 → Vehicle ID 1 → (120,210)

Frame 3 → Vehicle ID 1 → (145,220)

A lightweight centroid-based tracker can be used for simple traffic scenes. More complex scenes can use OpenCV-supported tracking methods.

6. Vehicle Counting

A virtual counting line is drawn across the road.

```
cv2.line(  
    frame,  
    (0,300),  
    (640,300),  
    (255,0,0),  
    2  
)
```

When a tracked vehicle crosses this line, the counter is increased.

Vehicle



Moves across frames



Crosses counting line



Check vehicle ID



Increase count

Using the vehicle ID prevents the same vehicle from being counted multiple times.

7. Visualization

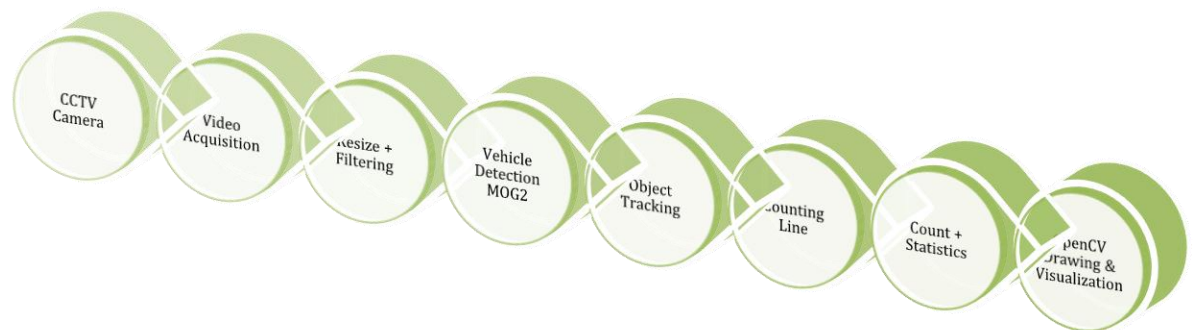
OpenCV drawing functions display useful information directly on the video.

```
cv2.rectangle(  
    frame,  
    (x,y),  
    (x+w,y+h),  
    (0,255,0),  
    2  
)  
  
cv2.putText(  
    frame,  
    "Vehicle",  
    (x,y-10),  
    cv2.FONT_HERSHEY_SIMPLEX,  
    0.6,
```

(0,255,0),
2
)
A statistics panel can display:

```
-----
TRAFFIC MONITOR
-----
Vehicles Detected : 24
Vehicles Counted : 18
Current FPS      : 28
-----
```

8. Complete Architecture



9. Accuracy and Real-Time Performance

Requirement	Solution
Vehicle detection	Background subtraction + contours
Movement tracking	Centroid/object tracking
Vehicle counting	Virtual line + unique vehicle IDs
Visualization	rectangle(), line(), putText()
Noise reduction	Gaussian blur + contour-area filtering
Real-time operation	640×480 resolution and lightweight algorithms
Continuous operation	Continuous CCTV frame processing
Avoid duplicate counting	Track unique vehicle IDs

Conclusion

The proposed system combines **OpenCV video acquisition, preprocessing, vehicle detection, tracking, counting, and visualization**. Background subtraction identifies moving vehicles, while tracking maintains their identity across frames. A virtual counting line determines when vehicles pass a specific point, and OpenCV drawing functions display bounding boxes, IDs, counting lines, FPS, and traffic statistics. Using reduced resolution, lightweight algorithms, and noise filtering ensures **accurate and continuous real-time traffic monitoring with minimal delay**.